

Complementi di Matematica e Calcolo Numerico A.A. 2024-2025

Laboratorio 3 – 10/3/2025

I numeri floating point

Per rappresentare su un calcolatore l'infinità dei numeri reali si utilizza un sottoinsieme di dimensione finita detto insieme dei numeri floating-point (virgola mobile) o numeri macchina. Matlab utilizza la rappresentazione in doppia precisione, con la quale è possibile rappresentare ogni numero reale x che si trova approssimativamente negli intervalli:

$$-1.80 \times 10^{308} \leq x \leq -2.23 \times 10^{-308}$$

$$2.23 \times 10^{-308} \leq x \leq 1.80 \times 10^{308}$$

Le variabili predefinite `realmax`, `realmin` di Matlab contengono il più grande ed il più piccolo numero macchina positivo rappresentabile in doppia precisione

```
>> realmax  
ans= 1.7977e+308  
>> realmin  
ans = 2.2251e-308
```

Nota: Ad ogni numero più grande di `realmax` Matlab associa il valore `Inf` o “infinito”. Quindi ad esempio:

```
>> 2*realmax
ans =
    Inf
```

Sarà **Inf** anche il risultato della valutazione di una espressione che contenga una divisione per 0 e senza generare un segnale di errore Matlab continuerà i calcoli!

Nota: Matlab riesce a rappresentare numeri più piccoli di **realmin** cambiando la rappresentazione ed utilizzando parte dei bit normalmente utilizzati per la mantissa per l'esponente. Questo comporta una perdita di cifre significative rispetto agli altri numeri macchina; se tale perdita supera un certo livello, allora Matlab restituisce semplicemente 0.

```
>> realmin/10
ans = 2.2251e-309
>> realmin/1e+16
ans = 0
```

Attenzione! a non confondere la rappresentazione interna di un numero floating point con il formato con cui tale numero viene visualizzato a video da Matlab in rappresentazione decimale.

Numeri reali come $1/3 = 0.\bar{3}$ $1/e = 0.367879441171442\dots$ vengono approssimati con numeri macchina che hanno un numero finito di cifre significative. Digitando

```
>> 1/3, 1/exp(1)
```

```
ans =
```

```
0.3333
```

```
ans =
```

```
0.3679
```

si visualizzano solo 4 cifre dopo la virgola.

Si può cambiare il formato di visualizzazione con il comando **format**.

Il formato di default che abbiamo appena visto è **format short**.

Altri formati possibili sono:

format long visualizza un numero reale con 15 cifre decimali

format short e visualizza un numero in notazione esponenziale con 5 cifre significative (es: 5.0000e-01 significa $5 * 10^{-1}$ ovvero 0.5)

format long e visualizza un numero in notazione esponenziale con 16 cifre significative.

ATTENZIONE la **e** che compare nella notazione esponenziale sta per 'elevato a', non ha nulla a che vedere con la costante di Nepero e !

Ad esempio

```
>> format long
```

```
>> 1/3
```

```
ans= 0.3333333333333333
```

```
>> 1/exp(1)
```

```
ans = 0.367879441171442
```

```
>> format short e
```

```
>> 1/3
```

```
ans = 3.3333e-01
```

```
>> 1/exp(1)
```

```
ans = 3.6788e-01
```

```
>> format long e
```

```
>> 1/3
```

```
ans = 3.333333333333333e-01
```

```
>> 1/exp(1)
```

```
ans = 3.678794411714423e-01
```

In Matlab è predefinita la variabile `eps` contenente il valore dell'epsilon macchina ϵ_M (il più piccolo numero tale che $\text{float}(1 + \text{eps}) > 1$).

```
>> eps
```

```
ans =
```

```
2.2204e-16
```

Cosa cambia in aritmetica finita?

Cancellazione numerica. È il fenomeno della perdita di cifre significative che si genera quando si sottraggono due numeri "quasi uguali". Ad esempio, dati $x = 0.765835896$ e $y = 0.765332562$, supponendo di lavorare con una aritmetica finita a 6 cifre significative, x e y saranno rappresentati da $\tilde{x} = 0.765835$ e $\tilde{y} = 0.765332$. Pertanto

$$\tilde{x} - \tilde{y} = 0.503000 \cdot 10^{-3}$$

mentre il valore esatto è

$$x - y = 0.503334 \cdot 10^{-3}$$

Esercizio 1 (Alcuni numeri non possono essere rappresentati)

Si ponga $a=1$ e $b=1e50$ (in notazione esponenziale $1e50$ significa 10^{50}). Si divida ripetutamente a per b e si osservi il risultato. Ripetere l'esercizio partendo da $a=1$ e moltiplicando per b .

Esercizio 2 (Esempio di cancellazione numerica)

In aritmetica esatta, usando la nota identità $(a+b)(a-b) = a^2 - b^2$, si ottiene facilmente

$$\begin{aligned} (\sqrt{x^2 + 1} - x)(\sqrt{x^2 + 1} + x) &= 1, \quad \implies \\ \sqrt{x^2 + 1} - x &= \frac{1}{\sqrt{x^2 + 1} + x} \quad \forall x \in \mathbb{R}. \end{aligned} \quad (1)$$

Calcolare con Matlab $y_1 = \sqrt{x^2 + 1} - x$ e $y_2 = \frac{1}{\sqrt{x^2 + 1} + x}$ per $x = 77777777$. Quale delle due è più accurata?

(R: $y_1 = 0$, $y_2 = 6.4286e - 09$. Il risultato “corretto” è y_2 , mentre il risultato dato da y_1 è soggetto al fenomeno di cancellazione numerica).

Esercizio 3 (Esempio di effetti dell’aritmetica finita)

In aritmetica esatta è ben noto il seguente limite

$$\lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n = e ,$$

dove $e = 2,718\dots$ rappresenta il numero di Nepero. Si calcoli e come `exp(1)`. Fissato un valore per n si calcoli $a_n = \left(1 + \frac{1}{n}\right)^n$ e l’errore assoluto $E_n = |a_n - e|$ commesso.

Per approssimare e si ripeta il calcolo per $n = 10^2, 10^4, 10^6, 10^8, 10^{12}, 10^{14}$. In aritmetica esatta, i valori E_n dovrebbero tendere a zero in quanto le a_n sopra calcolate tendono a e . Cosa accade invece?

Programmare con Matlab: Script-files

Che cos' è uno *script file*?

- È un file con estensione `.m` (ad esempio: `myfile.m`).
- Contiene una sequenza di istruzioni Matlab, scritte come se fossero digitate in modalità interattiva a destra del prompt.
- Digitando il nome (senza estensione `.m`) di uno *script-file* a destra del prompt:

```
>> myfile
```

vengono eseguite in successione tutte le istruzioni contenute nel file.

- Le variabili assegnate in uno *script-file* sono visibili nel Workspace, ovvero persistono in memoria al termine dell'esecuzione.

Alcune buone regole

- Il nome di uno *script-file* deve essere diverso dai nomi delle variabili che esso elabora e dai nomi delle variabili presenti in Workspace, altrimenti non verrà eseguito.
- Non assegnare ad uno *script-file* il nome di una funzione predefinita di Matlab. Per verificare se un nome esiste già:

```
>> exist('nome')
```

Se la risposta è diversa da 0, significa che il nome scelto è una variabile, o in file, o una function.

Matlab come linguaggio di programmazione

Come ogni linguaggio di programmazione Matlab fornisce gli operatori relazionali per confrontare numeri o il valore di espressioni numeriche

$==, \sim, <, >, \leq, \geq$

Per convenzione si associa il valore 1 a “vero” e 0 a “falso”.

$a == b \longrightarrow 1$ se $a = b$, 0 altrimenti

$a \sim b \longrightarrow 1$ se $a \neq b$, 0 altrimenti

Questi operatori si applicano anche a vettori, ad esempio

```
>> x = [4,1,19]; y = [5,0,3];
```

```
>> b = x <= y
```

```
>> b =
```

```
1 x 3 logical array
```

```
1    0    0
```

Si osservi che, sebbene sia costituito di 0 e di 1, b è definito come un vettore logico e non semplicemente come un vettore di numeri reali 0 ed 1.

Come ogni linguaggio di programmazione Matlab fornisce istruzioni apposite per realizzare controlli o cicli.

Se si vuole eseguire una o più operazioni solo nel caso in cui sia verificata una data condizione, si può utilizzare il comando `if`, che ha la seguente sintassi:

```
if (condizione==true)
    istruzioni
end
```

Ad esempio per controllare se la variabile x è non negativa prima di calcolarne la radice quadrata:

```
>> if(x>=0)
        r=sqrt(x);
>> end
```

Per ripetere un numero fissato di volte alcune istruzioni, si può realizzare un ciclo con il comando Matlab `for`, che ha la seguente sintassi:

```
for contatore=inizio:passo:fine
    istruzioni
end
```

Esempio: Dato un numero naturale positivo n , calcolare la matrice triangolare superiore A di ordine n definita da

$$A_{i,j} = \begin{cases} n & \text{se } i = j \\ i/(j+1) & \text{se } i < j \end{cases}$$

Ad esempio, se $n = 4$

$$A = \begin{bmatrix} 4 & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \\ 0 & 4 & \frac{2}{4} & \frac{2}{5} \\ 0 & 0 & 4 & \frac{3}{5} \\ 0 & 0 & 0 & 4 \end{bmatrix}$$

Per calcolare i coefficienti di A si potrebbe realizzare un doppio ciclo

```
for i = 1:n
    for j=i+1:n
        A(i,j)=i/(j+1);
    end
    A(i,i)=n;
end
```

In Matlab è buona regola, prima di scrivere un ciclo, vedere se è possibile evitarlo tramite un opportuno uso di istruzioni vettoriali.

Scriviamo uno *script-file* che, letta da tastiera la dimensione, calcoli la matrice triangolare superiore A nel modo seguente.

Per generare un nuovo *script-file* selezioniamo dal menu:

file $\longrightarrow$ new $\longrightarrow$ Mfile

`mat_trisup.m`

```
n=input('inserisci dimensione matrice ');
if(n<0)
    disp('dimensione della matrice inaccettabile');
    return;
end
A=eye(n)*n;
for i=1:n
    A(i,i+1:n)=i./[i+2:n+1];
end
A
```

Per eseguire il codice è sufficiente digitare `mat_trisup` a destra del prompt nella finestra di comando. Scegliendo come dimensione 4 otteniamo:

```
>> mat_trisup
inserisci dimensione matrice 4
A =
    4.00000    0.33333    0.25000    0.20000
    0.00000    4.00000    0.50000    0.40000
    0.00000    0.00000    4.00000    0.60000
    0.00000    0.00000    0.00000    4.00000
```

Ancora matrici

In Matlab è definita la funzione `inv` per calcolare l'inversa di una matrice quadrata di dimensione n .

Ad esempio se $B = \begin{bmatrix} 4 & -1 & 1 \\ -1 & 3 & -2 \\ 1 & -2 & 3 \end{bmatrix}$;

```
>> C=inv(B)
```

```
C =
```

```
    0.2778    0.0556   -0.0556  
    0.0556    0.6111    0.3889  
   -0.0556    0.3889    0.6111
```

Verifichiamo che il risultato ottenuto è effettivamente l'inversa controllando che il prodotto di B per la matrice calcolata C a destra o a sinistra restituisce sempre l'identità.

```
>> C*B
```

```
ans =
```

```
    1.0000         0     0.0000  
         0     1.0000         0  
         0         0     1.0000
```

```
>> B*C
```

```
ans =
```

```
    1.0000         0         0  
   -0.0000     1.0000         0  
    0.0000         0     1.0000
```

Si osserva che i coefficienti del prodotto non sono sempre esattamente 1 oppure 0 ma compaiono dei valori come 1.0000 o 0.0000. Questo significa che quel coefficiente non è esattamente 1 (o 0) ma una buona approssimazione. Visualizzando più cifre troveremmo che probabilmente intorno alla 15^{ma} cifra dopo la virgola c'è un coefficiente non nullo. Questo effetto è dovuto agli inevitabili errori di arrotondamento legati all'aritmetica finita di cui fa uso ogni calcolatore. Pertanto possiamo assumere che i prodotti calcolati $B \cdot C$ e $C \cdot B$ siano effettivamente la matrice identica.

Norme di matrici:

$$\|A\|_1 := \max_{j=1,\dots,n} \sum_{i=1}^n |a_{i,j}|$$

$$\|A\|_\infty := \max_{i=1,\dots,n} \sum_{j=1}^n |a_{i,j}|$$

Per calcolare $\|A\|_1$ in Matlab possiamo scrivere:

```
>> norm(A,1)      % oppure max(sum(abs(A)))
```

Per calcolare $\|A\|_\infty$ possiamo scrivere:

```
>> norm(A,inf)    % oppure max(sum(abs(A')))
```

Esercizio 4

Sia

$$A = \begin{bmatrix} 3 & 9 & 0 & -7 \\ 2 & 2 & 2 & 1 \\ 0 & 2 & 1 & 3 \\ 4 & -3 & 0 & 9 \end{bmatrix}$$

Calcolare $\|A\|_1$, $\|A^{-1}\|_\infty$ con il comando **norm** e, per conferma, con l'opportuna combinazione dei comandi **abs**, **max**, **sum**.

La funzione diag

La funzione `diag` di Matlab può essere applicata sia ad un vettore che ad una matrice e restituisce nei due casi oggetti differenti. Se applicata ad un vettore v di lunghezza n , genera una matrice diagonale di dimensione $n \times n$, contenente i coefficienti di v sulla diagonale principale e coefficienti nulli in tutte le altre posizioni. Ad esempio:

```
>> v=[1:4];  
>> A=diag(v)  
A =  
    1    0    0    0  
    0    2    0    0  
    0    0    3    0  
    0    0    0    4
```

Specificando un secondo argomento è possibile indicare su quale diagonale posizionare il vettore v . Ad esempio, per generare una matrice che abbia v sulla prima sopradiagonale scriviamo

```
>> A=diag(v,1)  
A =  
    0    1    0    0    0  
    0    0    2    0    0  
    0    0    0    3    0  
    0    0    0    0    4  
    0    0    0    0    0
```

Scrivendo

```
>> A=diag(v,-2)
```

A =

0	0	0	0	0	0
0	0	0	0	0	0
1	0	0	0	0	0
0	2	0	0	0	0
0	0	3	0	0	0
0	0	0	4	0	0

creiamo una matrice $(n+2) \times (n+2)$, ($n = 4$) che ha i coefficienti di v sulla seconda sottodiagonale. Si usano pertanto indici positivi per le sopradiagonali e negativi per le sottodiagonali.

Se applichiamo **diag** ad una matrice l'effetto sarà quello opposto, ovvero otterremo il vettore dei coefficienti della matrice che si trovano sulla diagonale principale se non diversamente specificato, oppure su una sopra o sotto diagonale indicata.

Ad esempio

```
>> M=[1 2 9; 7 5 6; 4 8 3]
```

```
>> M=
```

1	2	9
7	5	6
4	8	3

```
>> v=diag(M)
```



```

v =
    1
    5
    3
>> w=diag(M,1)
w =
    2
    6

```

Esempio. Utilizzando la funzione **diag**, costruiamo la matrice 7×7 che ha tutti i coefficienti uguali a 2 sulla diagonale principale e uguali a -1 sulle prima sopra e sotto diagonali.

```

>> A=diag(2*ones(7,1))-diag(ones(6,1),1)-diag(ones(6,1),-1)
A =
    2    -1     0     0     0     0     0
   -1     2    -1     0     0     0     0
    0    -1     2    -1     0     0     0
    0     0    -1     2    -1     0     0
    0     0     0    -1     2    -1     0
    0     0     0     0    -1     2    -1
    0     0     0     0     0    -1     2

```

Esercizio 5

1. Sia v il vettore generato con il comando $v=[1:0.3:2.2]$.
Calcolare: $\text{diag}(v), \text{diag}(v,1), \text{diag}(v,-1), \text{diag}(v,3),$
 $\text{diag}(v,-2)$.
2. Utilizzando la funzione **diag**, costruire le seguenti matrici 10×10 .

$$\begin{bmatrix} 1 & 0 & -1 & 0 & \dots & 0 \\ -1 & 2 & 0 & -1 & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & \dots & -1 & 8 & 0 & -1 \\ 0 & \dots & \dots & -1 & 9 & 0 \\ 0 & \dots & \dots & \dots & -1 & 10 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0.1 & 0.1 & \dots & \dots & 0.1 \\ 1 & 4 & 0 & \dots & \dots & 0 \\ 1 & 0 & 9 & 0 & \dots & 0 \\ 1 & \dots & \dots & \dots & \dots & \dots \\ 1 & 0 & \dots & \dots & 81 & 0 \\ 1 & 0 & \dots & \dots & \dots & 100 \end{bmatrix}$$

Generare matrici triangolari con `tril` e `triu`

Le funzioni predefinite di matlab `tril` e `triu` permettono di costruire matrici triangolari inferiori e superiori, rispettivamente, e funzionano in modo molto simile alla funzione `diag`.

```
>> C=[4 -1 1 5;-1 3 -2 3; 1 -2 3 9; 1 3 7 6];
```

```
C =
```

```
     4     -1      1      5
    -1      3     -2      3
     1     -2      3      9
     1      3      7      6
```

```
>> L=tril(C)
```

```
L =
```

```
     4      0      0      0
    -1      3      0      0
     1     -2      3      0
     1      3      7      6
```

```
>> L=tril(C,-1)
```

```
L =
```

```
     0      0      0      0
    -1      0      0      0
     1     -2      0      0
     1      3      7      0
```

```
>> U= triu(C)
U =
     4     -1      1      5
     0      3     -2      3
     0      0      3      9
     0      0      0      6

>> U=triu(C,+1)
U =
     0     -1      1      5
     0      0     -2      3
     0      0      0      9
     0      0      0      0
```

Esercizio 6

Sia A la matrice di Hilbert 4×4 . Estrarre da A le matrici triangolari inferiore e superiore aventi sotto, e rispettivamente sopra la diagonale, gli stessi elementi di A .

Esercizio 7

Si trasformi la matrice **A=ones(10)** in una matrice con 2 sottodiazionali e 4 sopradiagonali non nulle utilizzando una combinazione dei comandi **tril** e **triu**.

Sistemi lineari

Siano $A \in \mathbb{R}^{n \times n}$ una matrice quadrata non singolare ($\det(A) \neq 0$) e $b \in \mathbb{R}^{n \times 1}$ un vettore assegnati, allora esiste un unico vettore $x \in \mathbb{R}^{n \times 1}$ soluzione del sistema lineare

$$Ax = b$$

In Matlab, una volta assegnati A e b , si può calcolare la soluzione x del sistema con l'operatore $\backslash$.

```
>> x = A\b
```

Esercizio 8

Risolvere il seguente sistema lineare con l'operatore $\backslash$:

$$\begin{bmatrix} -16 & 8 & -7 \\ 1 & -5 & -3 \\ 1 & 10 & 14 \end{bmatrix} x = \begin{bmatrix} -9 \\ 4 \\ -13 \end{bmatrix} \longrightarrow x = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

Visualizzare i risultati in **format long e** e confrontarli con quelli che si ottengono dal prodotto dell'inversa di A con il vettore b ossia

```
>> x = inv(A)*b
```

Perchè i risultati non coincidono? Perchè non si usa questa seconda strategia?

Esercizi di riepilogo

Scrivere uno *script file* in cui si risolvono gli esercizi seguenti:

1. In aritmetica esatta vale l'identità

$$\frac{(x+1)-1}{x} = 1 \quad \forall x \neq 0;$$

valutare la quantità a sinistra dell'uguale per i seguenti valori di x : **1e-08, 1e-10, 1e-12, 1e-14** e calcolare l'errore rispetto al valore esatto 1. Cosa si osserva?

2. Costruire la matrice A di dimensione 10×10 avente come soli elementi non nulli, quelli della diagonale principale uguali ad 20, quelli delle prime sopra e sottodiagonali uguali ai primi numeri naturali positivi, e quelli della terza sopra e sottodiagonale uguali a -2 :

$$A = \begin{bmatrix} 20 & 1 & 0 & -2 & .. & 0 \\ 1 & 20 & 2 & 0 & -2 & 0 \\ 0 & 2 & 20 & 3 & 0 & 0 \\ -2 & 0 & 3 & 20 & 4 & .. \\ 0 & -2 & 0 & 4 & 20 & \\ .. & & .. & & & -2 \\ .. & & .. & & & .. \\ .. & & & & & 10 \\ 0 & .. & & 0 & -2 & .. & 10 & 20 \end{bmatrix}.$$

3. Si costruisca la matrice A che ha coefficienti uguali a 2 sulla diagonale principale, uguali a -1 sotto la diagonale e uguali a 1 sopra la diagonale in due modi differenti: a partire dalla matrice diagonale con coefficienti uguali a 2, utilizzando poi un singolo ciclo **for**, oppure sfruttando i comandi **tril** e **triu**. Si trasformi infine la matrice in una matrice che ha solo 5 sotto e sopradiagonali non nulle.

4. Data la matrice

$$A = \begin{bmatrix} -5 & 8 & -7 & 1 \\ 12 & -5 & -3 & 1 \\ 1 & 10 & 14 & 1 \\ 1 & 0 & 1 & 2 \end{bmatrix},$$

ed i termini noti

$$b_1 = \begin{bmatrix} -43 \\ 21 \\ -1 \\ 3 \end{bmatrix}, \quad b_2 = \begin{bmatrix} -1 \\ 76 \\ -26 \\ 3 \end{bmatrix},$$

si risolvano i sistemi lineari $Ax_i = b_i$ per $i = 1, 2$. (Soluzioni $x_1 = [1, -3, 2, 0]^T$, $x_2 = [6, 1, -3, 0]^T$).